Practical Exercise 3: AWS Database Services

This exercise explores what NoSQL is and provides you with an example of the NoSQL architecture to help you to understand how data is stored in NoSQL datastores differently to SQL datastores. We will also be exploring the benefits and reasons to use NoSQL datastores, as well as discuss a few examples of when you would use them.

Amazon DynamoDB is a fully managed NoSQL database service by Amazon. It is key-value store and designed to address the core problems of database management, performance, scalability, and reliability. DynamoDB is fast and provides predictable performance using SSDs (Solid State Drives) and you can easily provision and change the request capacity needed for each table.

The data model in a typical relational database has a predefined schema/table name, primary keys, list of its column names and their data types. All records stored in each table within such a database must have the same set of columns. However, **DynamoDB** (which is a **non-relational database**) only requires that a table (or a collections of items) has a primary key and does not require you to define all the attribute names (i.e. columns in the relational world) and data types in advance. A record of data (often referred to as a row in the relational world) stored in DynamoDB is called an item. An item can have any number of attributes. Each attribute, however, has a limit of 400KB for each item.

Activity 1: Create a DynamoDB Table

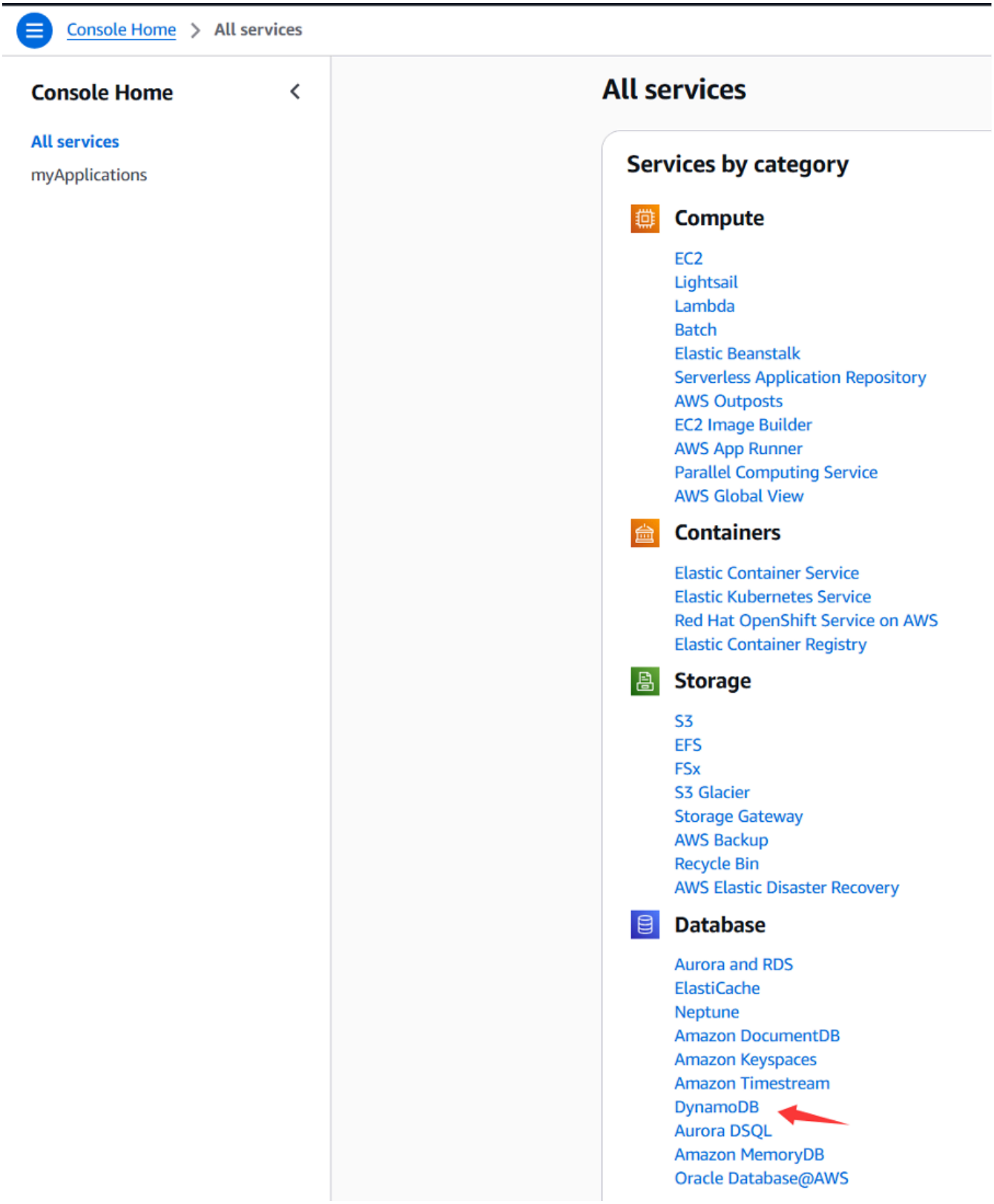
As part of this activity, you will be interacting with DynamoDB using the AWS DynamoDB Management Console.

For all the tasks in this activity, we will be working with the following dataset:

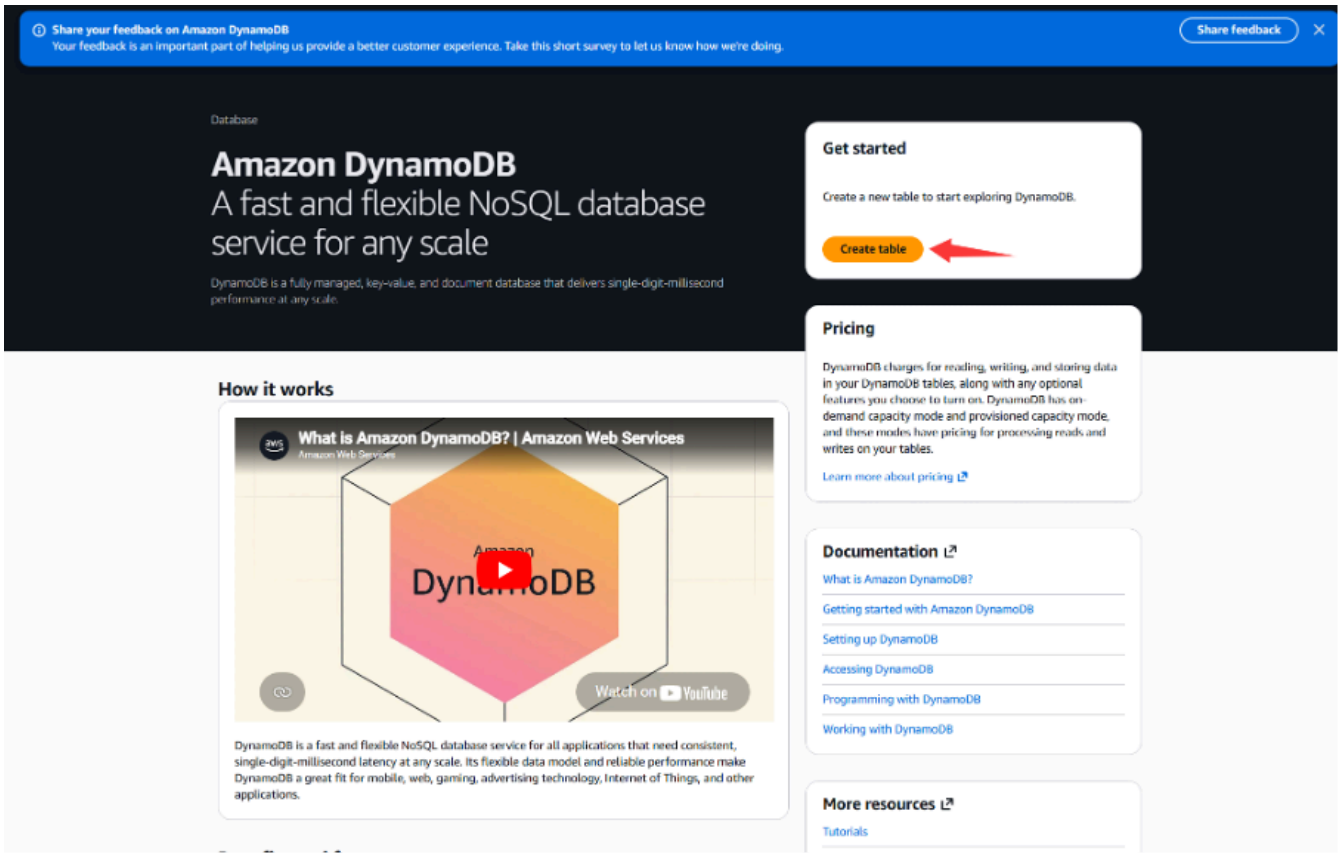
Artist	SongTitle	AlbumTitle	Year
No One You Know	Call me Today	Somewhat Famous	2015
No One You Know	My Dog Spot	Key Now	2014
No One You Know	Somewhere Down the Road	Somewhat Famous	2015

Follow the instructions below to create a new table.

Step 1: In the AWS Management Console, click **Services** and then click **DynamoDB** under the **Database** category.



Step 2: On the **DynamoDB** dashboard, click **Create table**.

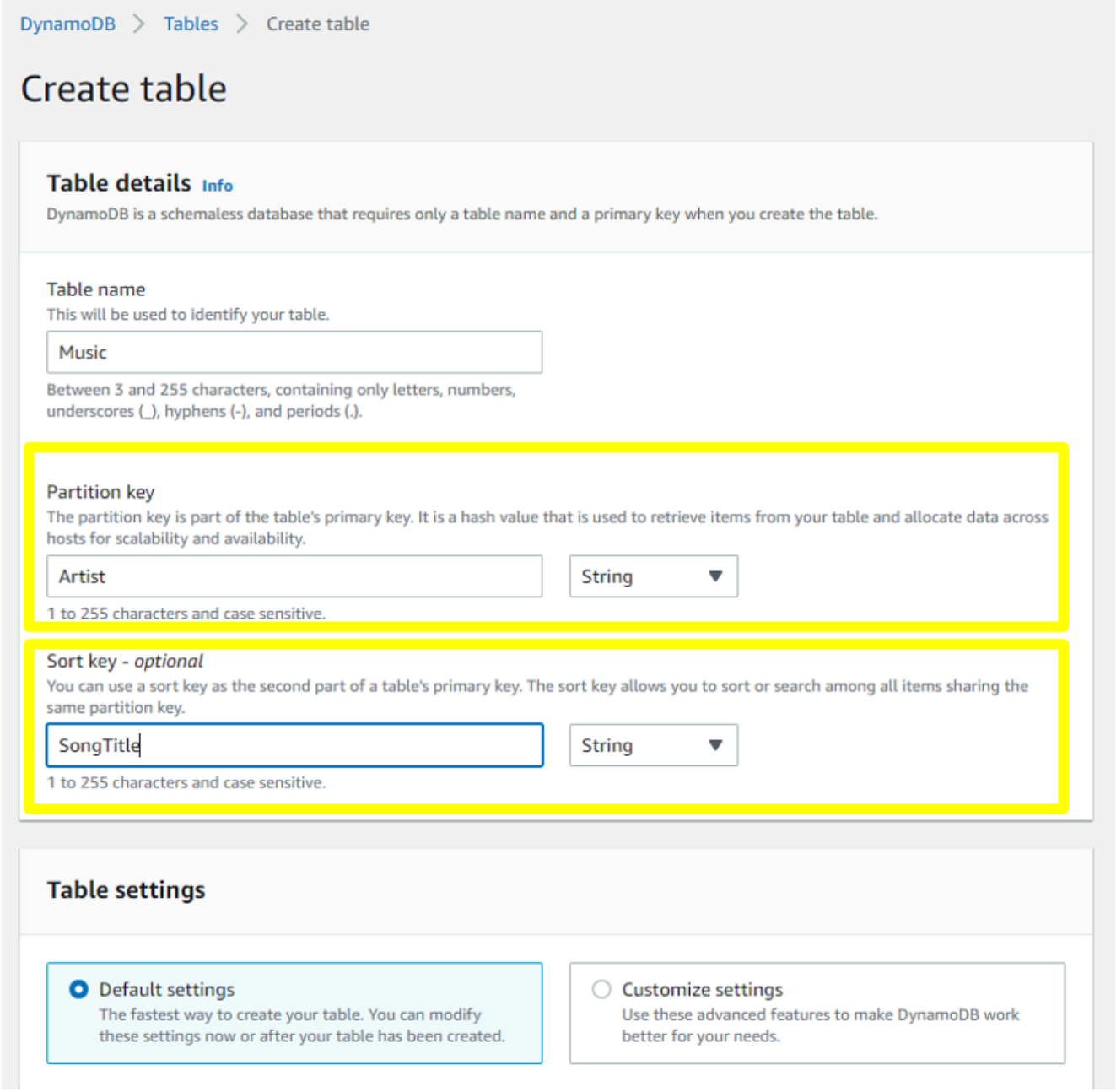


Step 3: In the **Create table** page, enter **Music** as the “Table name”.

Step 4: For **Partition key**, type **Artist** and select **String** as the data type.

Step 5: For **Sort key**, type **SongTitle** and leave **String** selected as the data type.

Step 6: In the table **Settings** section, leave **Default Settings** selected and click **Create table** at the bottom. Now your page should be like as below:

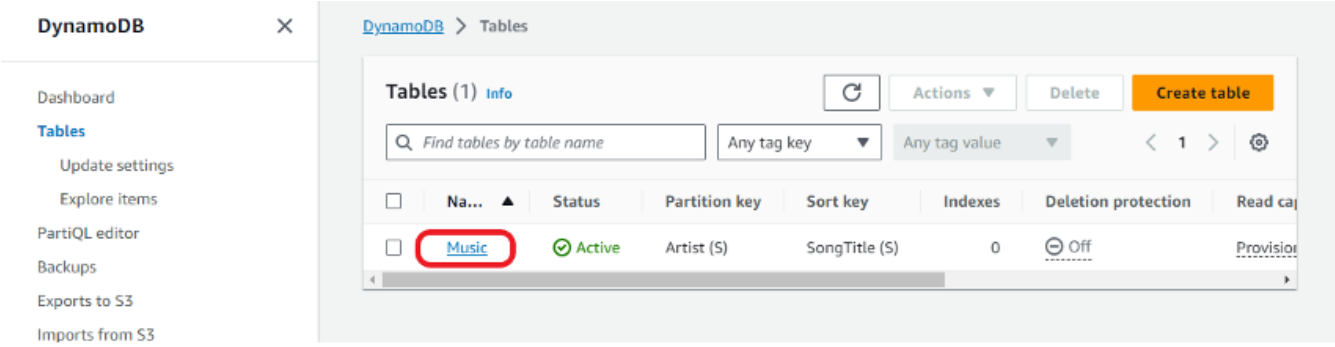


You should have been able to create the DynamoDB table by now. If not, please go back and check that you have followed all the steps accurately. If you hit roadblock, ask your tutor to assist you.

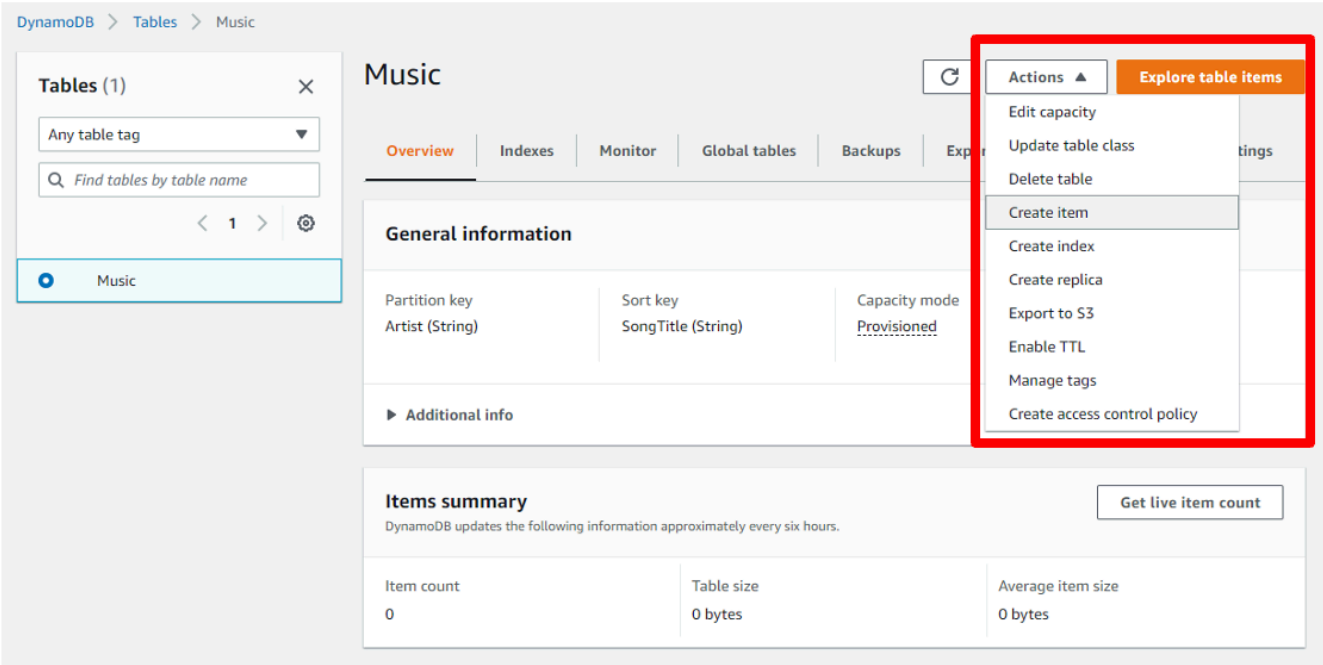
Activity 2: Adding Data in DynamoDB

Once the Music table has successfully been created in the previous activity, do the following steps to add data.

Step 1: Select **Music** table from the **Tables** view on the DynamoDB Management Console, as below.



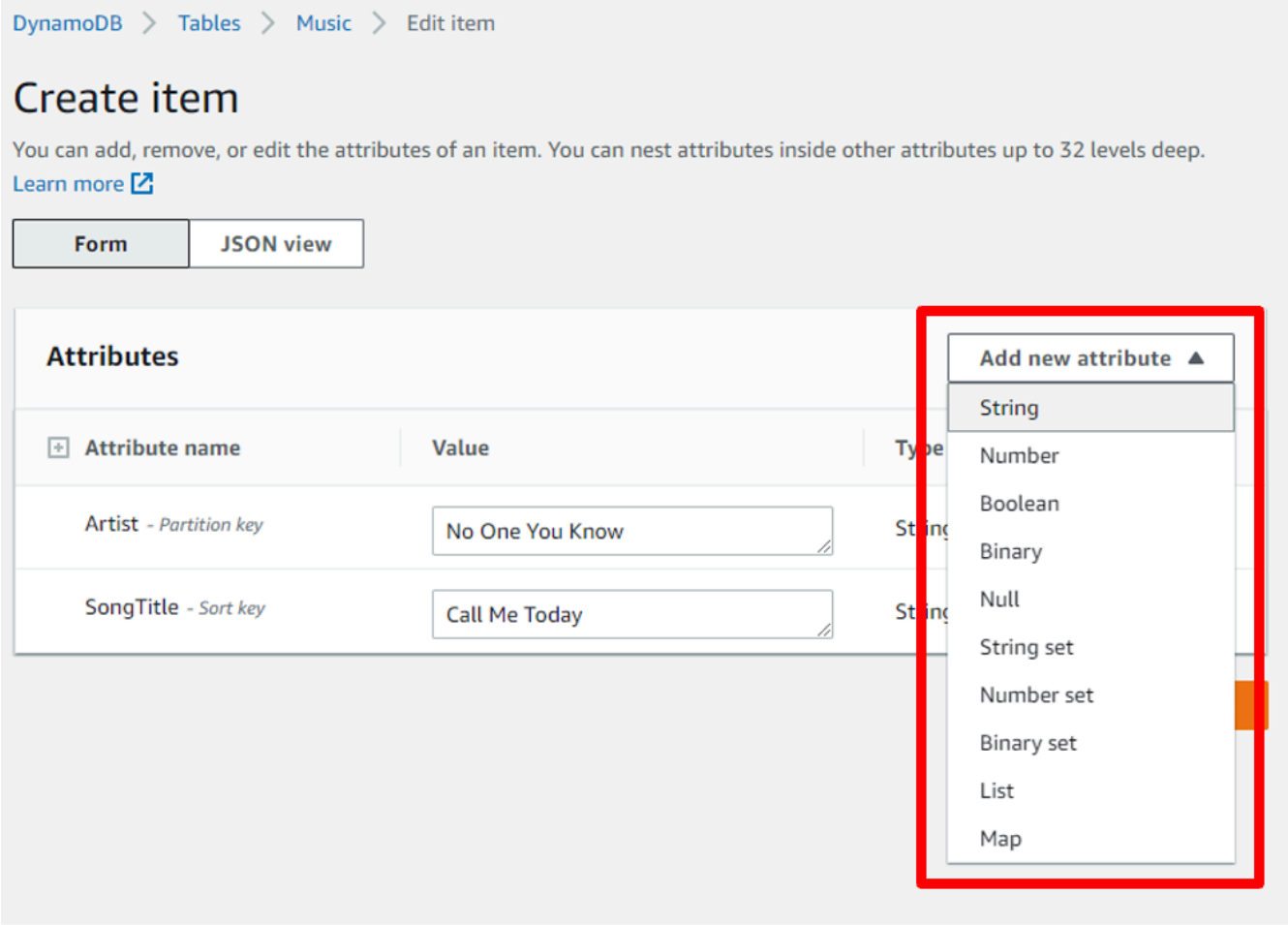
Step 2: Click on **Actions** drop-down box and select **Create item**.



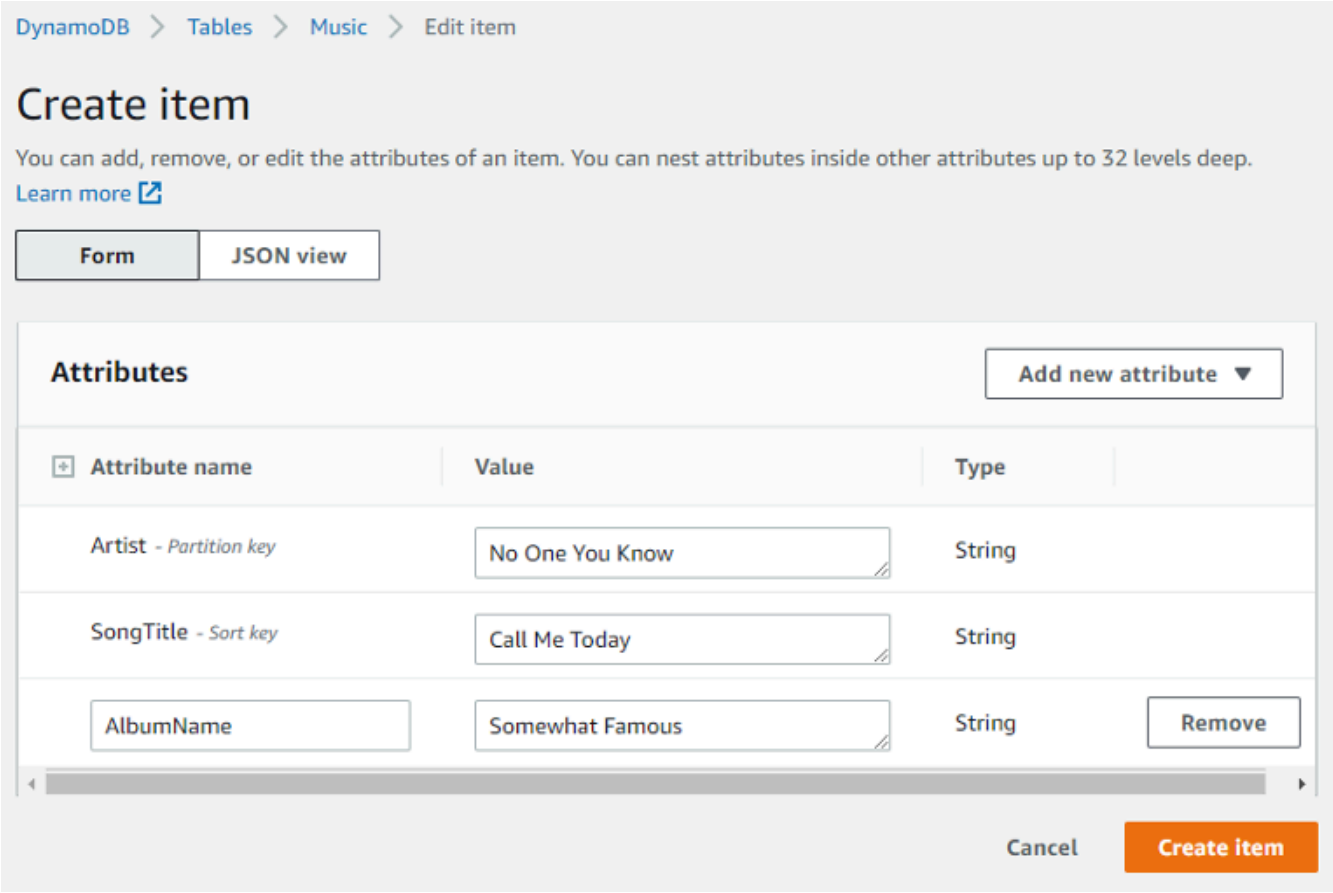
Step 3: In the page that pops up, enter **No One You Know** as the value of the **Artist** (which is of the type **String**) attribute in the **Value** field.

Step 4: For the **SongTitle** attribute (also of type **String**), enter **Call Me Today** in the **Value** field.

Step 5: To create an additional attribute, click the **Add new attribute** drop-down box to the right, and select its type as **String** and a row representing the new attribute row will be added.



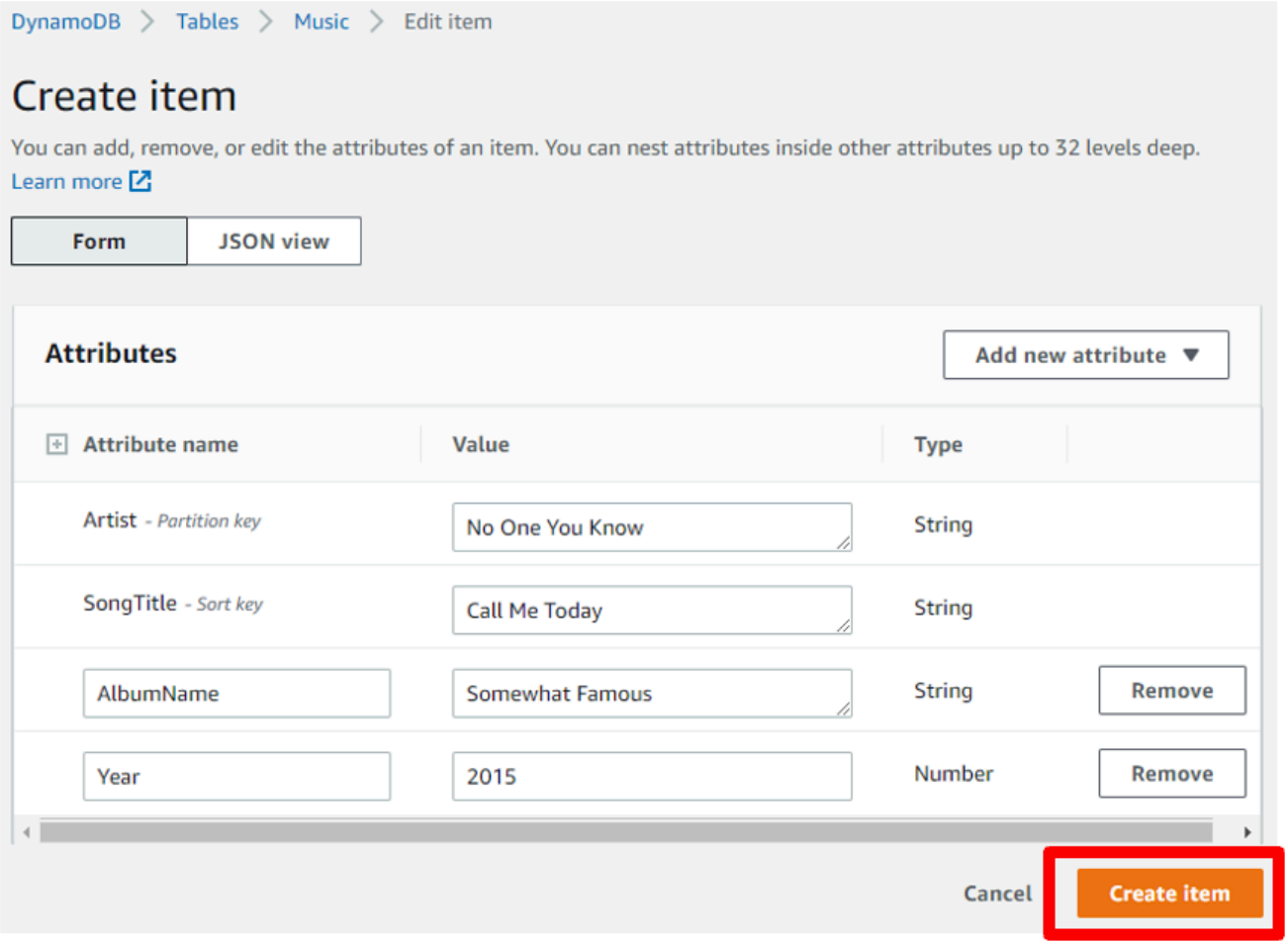
Step 6: Enter **AlbumName** as the **Attribute name**, and **Somewhat Famous** as the **Value** in the appropriate text boxes.



Step 7: To create another new attribute, follow Step 4 again.

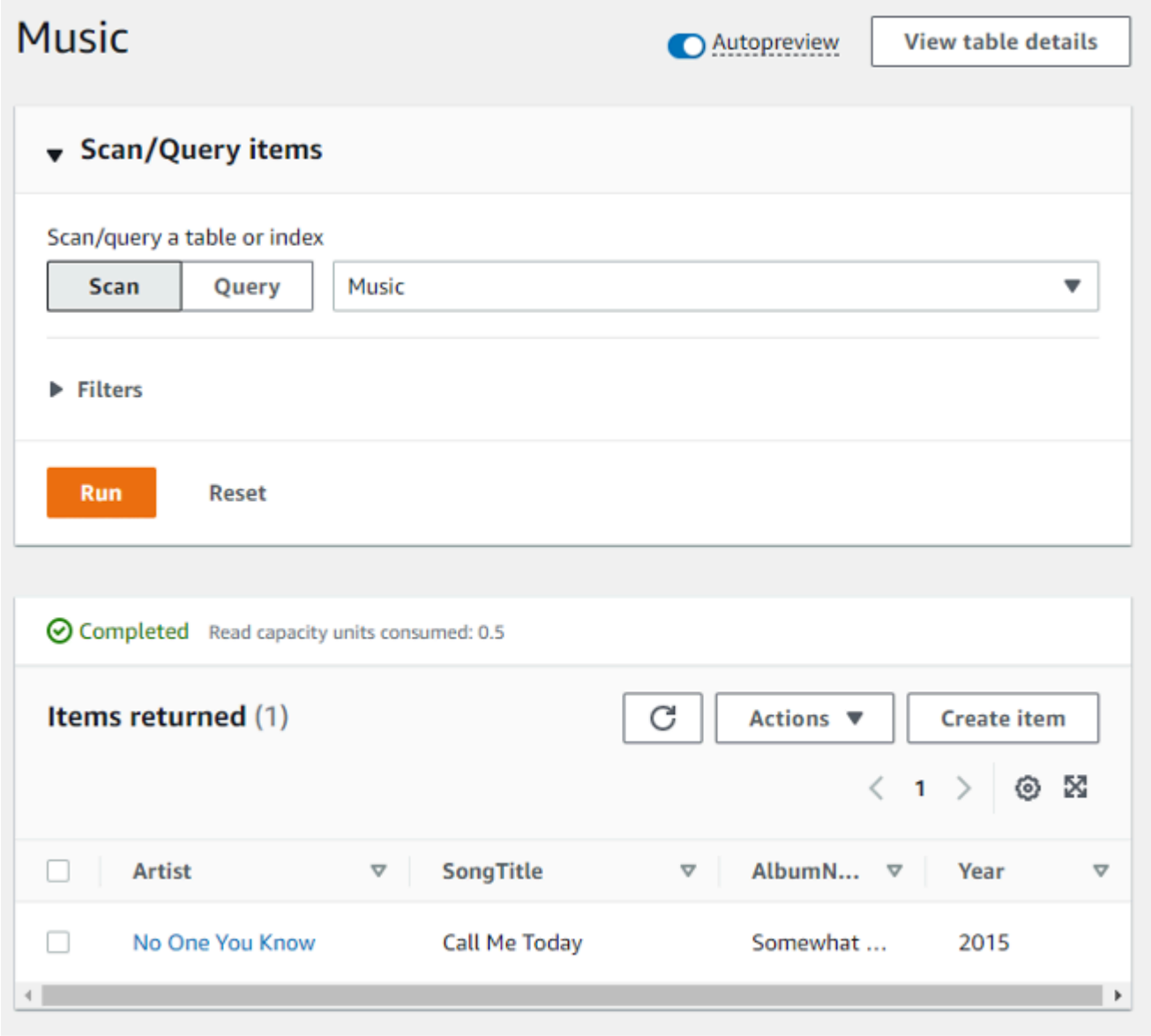
Step 8: This time, in the drop-down list, select **Number** as the type of the attribute, and then, the new attribute will be added to the table.

Step 9: For the new attribute, enter **Year** and **2015** in the **Attribute name** and **Value** boxes, respectively.



Step 10: Click **Create item** to store the new item with its four attributes in the **Music** table.

Step 11: Select the **Explore table item,** on the **Items** tab**,** you can view the list of items store in the **Music** table (see the screenshot on the right).

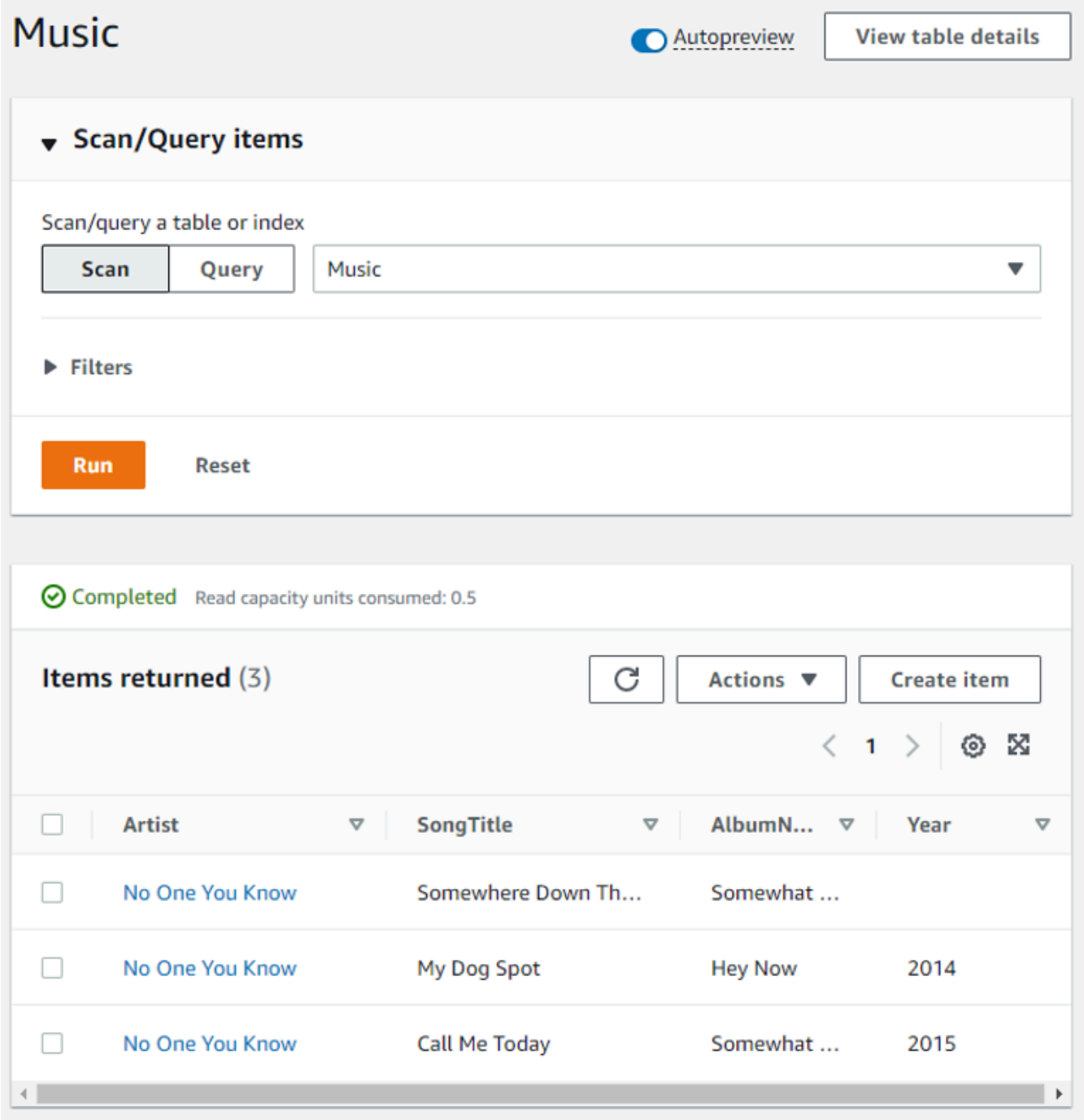


Step 12: Now, create second and third items with the following details.

Attribute Name	Attribute Type	Attribute Value
Artist	String	No One You Know
SongTitle	String	My Dog Spot
AlbumTitle	String	Hey Now
Year	Number	2014

Attribute Name	Attribute Type	Attribute Value
Artist	String	No One You Know
SongTitle	String	Somewhere Down The Road
AlbumTitle	String	Somewhat Famous

Step 13: After creating two more items as above, you will see the table contents as below.



You should have now been able to create items in the DynamoDB table we created in the previous activity. If not, please go back and check that you have followed the steps accurately. If you hit a roadblock, do seek assistance from your tutor.

Activity 3: Modifying Data in DynamoDB

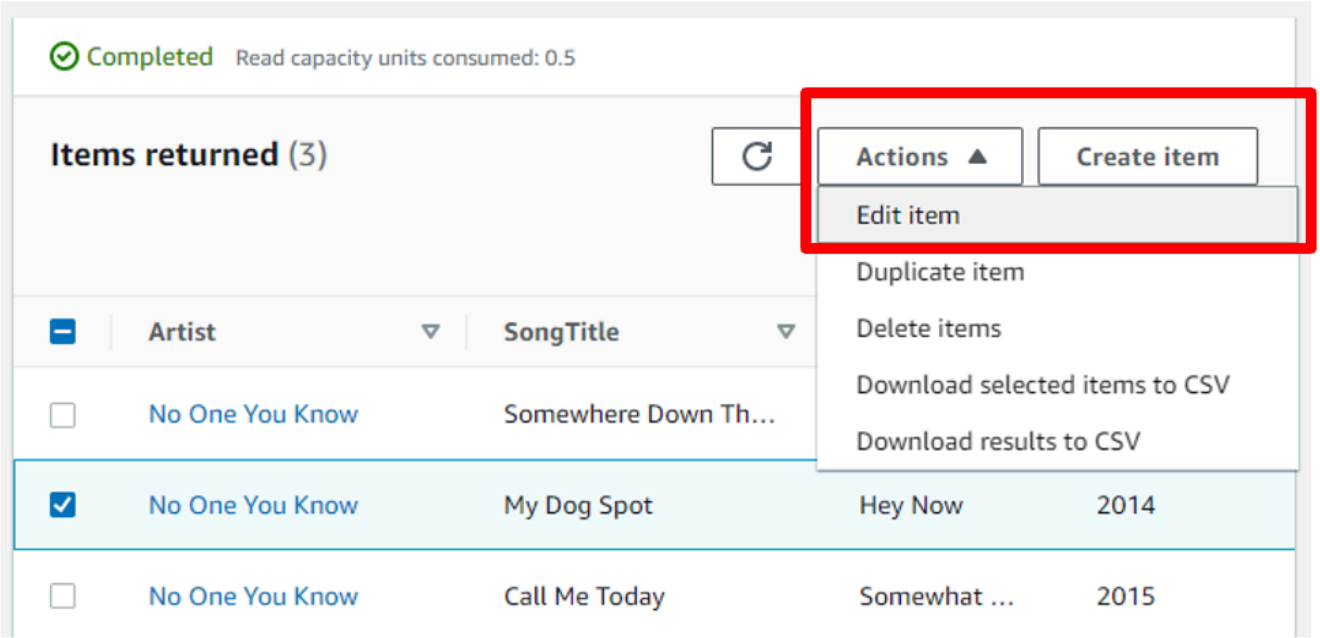
Follow the instructions below to learn how to modify an existing item in a DynamoDB table.

Step 1: Click **Tables** from the left navigation menu to return to the list of tables.

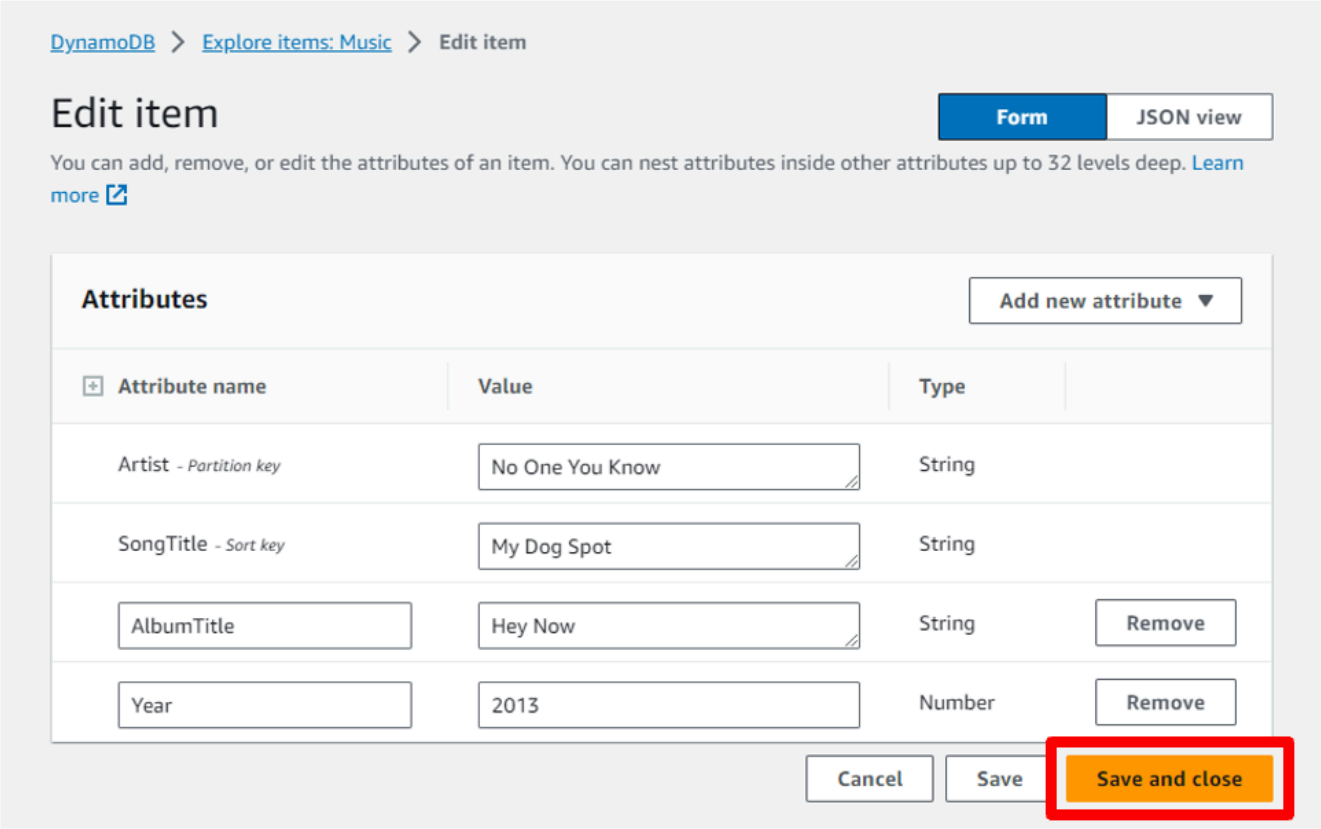
Step 2: Click the link **Music** to open the table view associated with the **Music** table.

Step 3: Click the **Explore table items** to open the "Scan/Query items" wizard as well as list out items already added to the **Music** table.

Step 4: From the list of items displayed, select the item with "Year" carrying the value "2014" to open **Item editor**. Alternatively, you can also click **Actions** and select **Edit item**, which would also take you to the **Item editor**.



Step 5: While on the **Edit item**, change the **Year** attribute value from 2014 to 2013. Then click **Save and close** to conclude this task.


DynamoDB > Explore items: Music > Edit item
Edit item
Form
JSON view
You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. Learn more
Attributes
Add new attribute
Attribute name
Value
Type
Artist - Partition key
No One You Know
String
SongTitle - Sort key
My Dog Spot
String
AlbumTitle
Hey Now
String
Remove
Year
2013
Number
Remove
Cancel
Save
Save and close

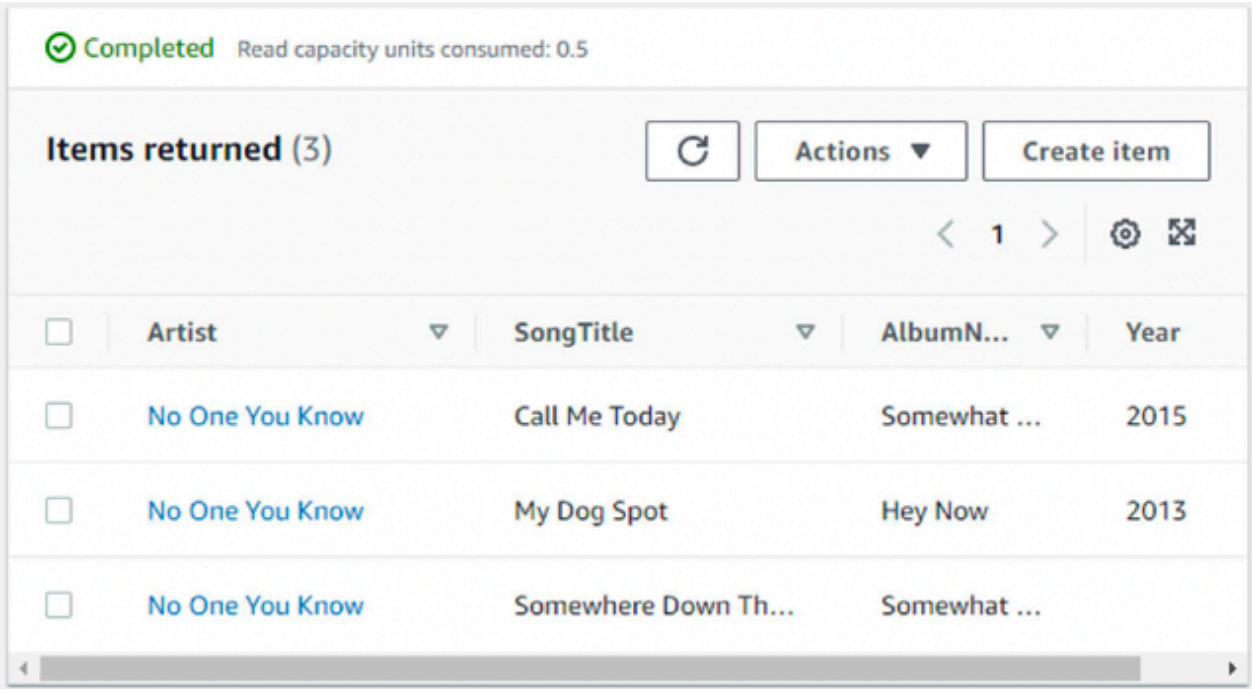
Activity 4: Querying DynamoDB Tables

This task provides examples of the following query operations. All these example queries are specified and run against the Music table. Remember, the primary key of this table is made of Artist (**partition** key) and SongTitle (**sort** key).

4.1 Query Using Only the Partition Key

For example, find songs by an artist.

1. Click **Tables** from the left navigation menu to return to the list of tables.
2. Click the link **Music** to open the table view associated with the Music table.
3. Click the **Explore table items** to open the “Scan/Query items” wizard as well as list out items already added to the Music table.
4. From “Scan/Query items” section, click on the button named “Query”, which toggles the view and enable “Query” dialog.
5. Type “**No One You Know**” in the **Partition key** field and click **Run**.
6. You will see the list of results (i.e. all three records we have added to the table thus far), which has the artist’s name as **No One You Know** in the **Items returned** section.

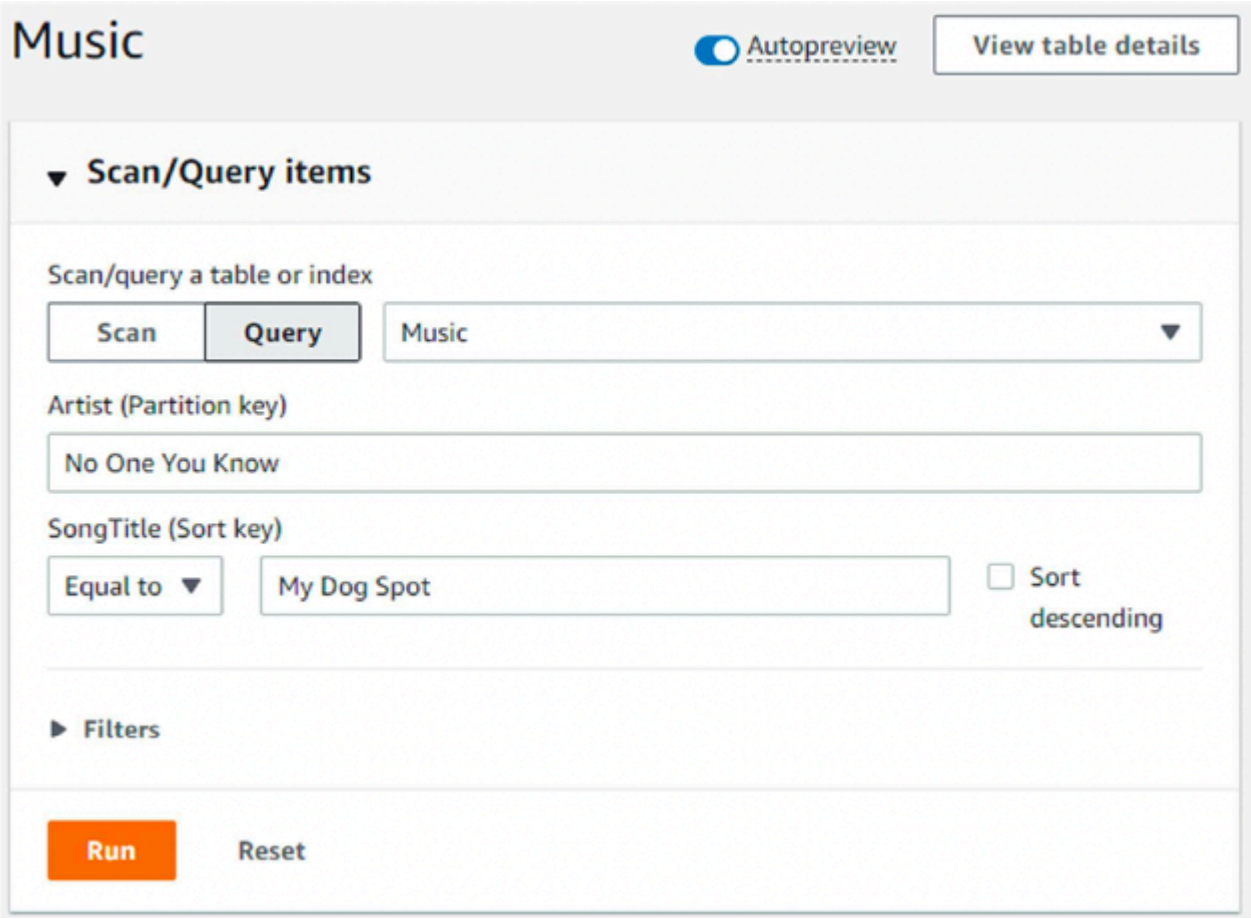


Did you notice that the returned results are sorted by the "SongTitle" (sort key) attribute?

4.2 Query Using Both the Partition Key and the Sort Key

For example, find songs by an artist and song title starting with a specific string.

Type "**My Dog Spot**" in the **Sort Key** field, in addition to the previously configured **PartitionKey** field**.**



The tracks for which the song title is "My Dog Spot" Will be displayed.

Great work! You've now successfully queried a DynamoDB table. You can also add filters to the scan operation by clicking on the **Add Filter** button, and then by providing the filtering criteria.

Reflection on your experience:

- Were there any issues that you have come across?
- Was it straight-forward?
- What did you learn from this experience and from Amazon Dynamo?

- Were there any shortcuts or guides that you came across that were helpful?

Activity 5: Working with Complex Data Types

The aim of this activity is to help you learn to create, deploy, query, return data stored on Amazon DynamoDB of simple data types to complex objects encoded in JSON. You will use the AWS SDK for Java to write simple programs to perform the following Amazon DynamoDB operations:

- Create a table called **Movies** and load a sample dataset in JSON format.
- Perform create, read, update, and delete operations on the table.
- Run simple queries.

5.1 Setting Up and Starting with AWS SDK for Java on Amazon DynamoDB

The AWS SDK for Java offers several programming models for different use cases. In this task, the sample Java code provided uses the document model that provides a level of abstraction that makes it easier for you to work with JSON documents.

Follow the instructions below to help you get started.

In the following exercise, we will be using “DynamoDB Local”, which is a tool provided by AWS for developers to test their DynamoDB applications locally in their development environments without connecting to the DynamoDB web service.

1. Download dynamodb_local_latest.zip from **Week Module.**
2. Extract the contents, and copy the extracted directory to a location of your choice.
3. To start DynamoDB on your computer, open a command prompt window, navigate to the directory where you extracted **DynamoDBLocal.jar** and type the following command. (This step assumes that you already have Java installed in your local development environment.)

```
java -Djava.library.path=./DynamoDBLocal_lib -jar Dynam
oDBLocal.jar -sharedDb
```

```
C:\dynamodb_local_latest>java -Djava.library.path=./DynamoDBLocal_lib -jar DynamoDBLocal.jar -sharedDb
Initializing DynamoDB Local with the following configuration:
Port:   8000
InMemory:       false
DbPath: null
SharedDb:       true
shouldDelayTransientStatuses:   false
CorsParams:     *
```

Note: DynamoDB Local, **by default, runs on port 8000.** Therefore, your local development environment already has a process running on the port 8000, you might run into an error shown below:

```
H:\Work\RMIT\OUA\2022\SP3\Tutorials\Week6\dynamodb_local_latest>java -Djava.library.path=./DynamoDBLocal_lib -jar Dynamo
DBLocal.jar -sharedDb
Initializing DynamoDB Local with the following configuration:
Port:   8000
InMemory:       false
DbPath: null
SharedDb:       true
shouldDelayTransientStatuses:   false
CorsParams:     null

Exception in thread "main" java.io.IOException: Failed to bind to 0.0.0.0/0.0.0.0:8000
        at org.eclipse.jetty.server.ServerConnector.openAcceptChannel(ServerConnector.java:349)
        at org.eclipse.jetty.server.ServerConnector.open(ServerConnector.java:310)
        at org.eclipse.jetty.server.AbstractNetworkConnector.doStart(AbstractNetworkConnector.java:80)
        at org.eclipse.jetty.server.ServerConnector.doStart(ServerConnector.java:234)
        at org.eclipse.jetty.util.component.AbstractLifeCycle.start(AbstractLifeCycle.java:73)
        at org.eclipse.jetty.server.Server.doStart(Server.java:401)
        at org.eclipse.jetty.util.component.AbstractLifeCycle.start(AbstractLifeCycle.java:73)
        at com.amazonaws.services.dynamodbv2.local.server.DynamoDBProxyServer.start(DynamoDBProxyServer.java:62)
        at com.amazonaws.services.dynamodbv2.local.main.ServerRunner.main(ServerRunner.java:76)
Caused by: java.net.BindException: Address already in use: bind
        at sun.nio.ch.Net.bind0(Native Method)
        at sun.nio.ch.Net.bind(Unknown Source)
        at sun.nio.ch.Net.bind(Unknown Source)
        at sun.nio.ch.ServerSocketChannelImpl.bind(Unknown Source)
        at sun.nio.ch.ServerSocketAdaptor.bind(Unknown Source)
        at org.eclipse.jetty.server.ServerConnector.openAcceptChannel(ServerConnector.java:344)
        ... 8 more
```

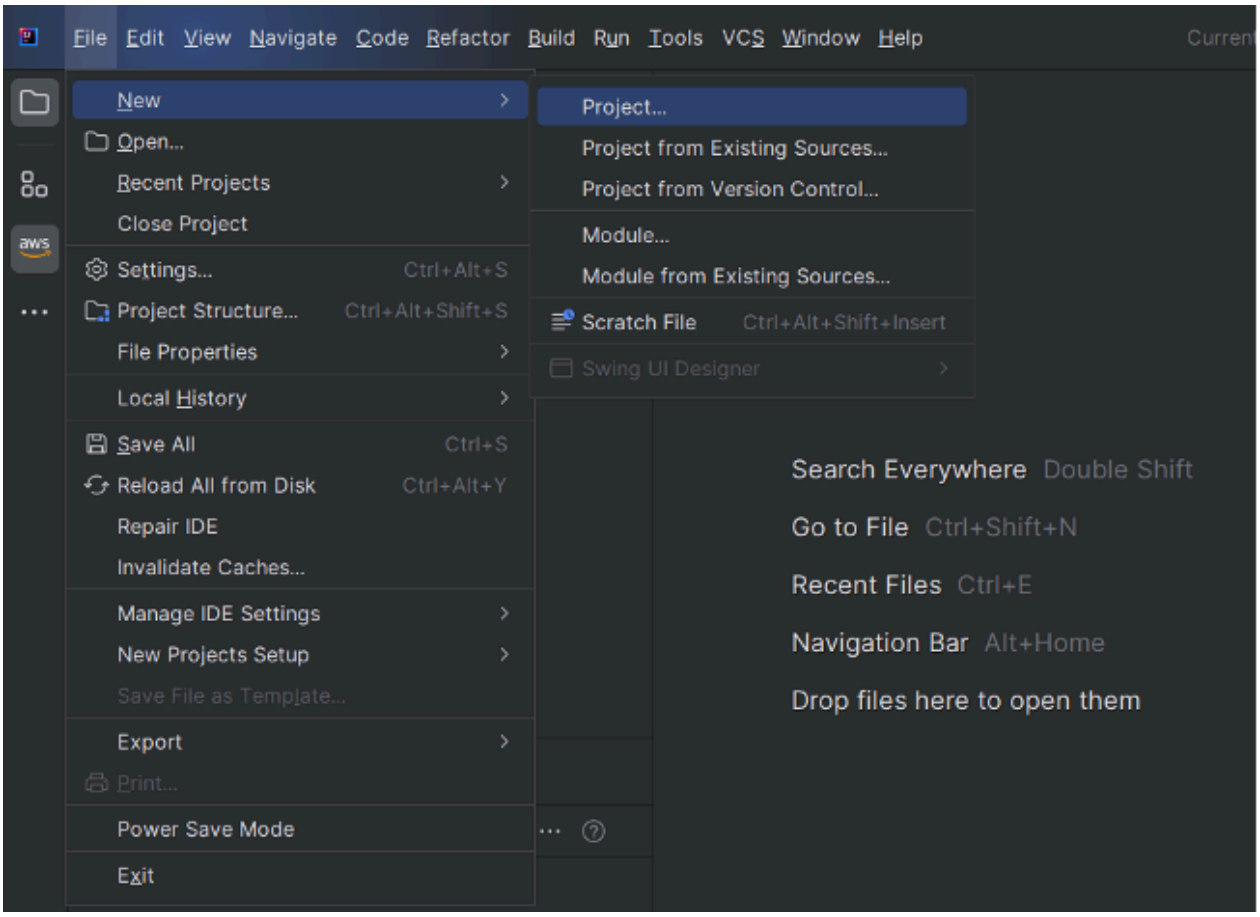
To avoid this, you can specify a different port for DynamoDB Local to run using the -port directive, as below.

```
java -Djava.library.path=./DynamoDBLocal_lib -jar DynamoDBLocal.jar -sharedDb -port [your preferred port number - e.g. 8001]
```

Note: If you do go ahead with specifying an alternative port number, make sure to change your DynamoDB endpoint URL to reflect that in the code samples used in the subsequent activities.

4. Make sure you have set up the AWS Toolkit on to your preferred IDE for Java (e.g. IntelliJ IDEA**). You should be fine if you followed the practical exercises in previous weeks.**
5. From the menu bar at the top, select **File > New > Project** to create a new Java project. (**Note:** you will first need to tap the "≡" to reveal the

menu bar at the top.)



6. In the Project name field, type a name for your project (e.g. **DynamoDBTasks**).

   Select Language as **Java**, Build system as **Maven**, JDK as **1.8**. Next, untick the option **Add sample code**.

   Now, expand the **Advanced Settings** section, and configure **GroupId** as **com.amazonaws.samples**.

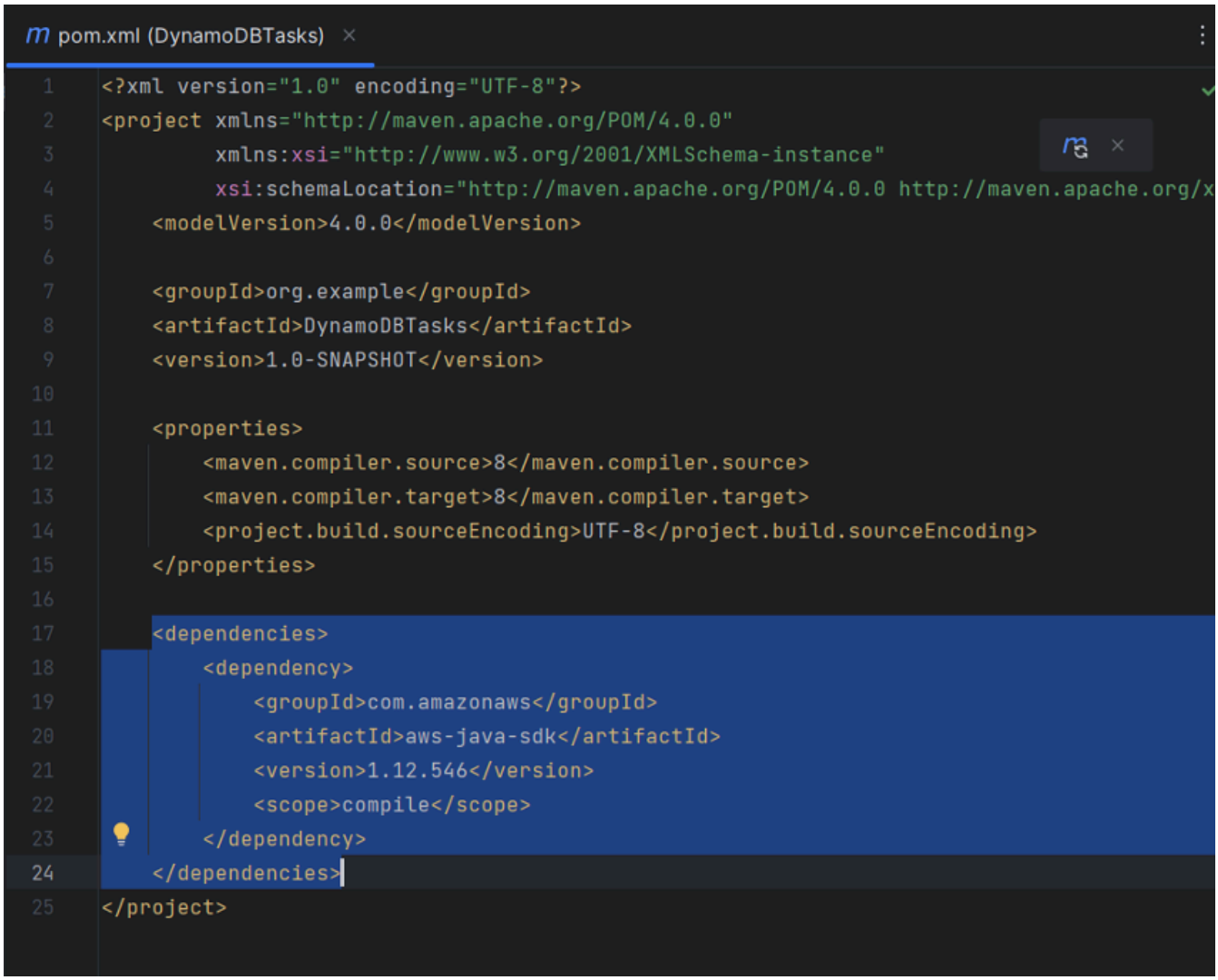
   Finally, tap the **Create** button at the bottom of the screen to create the project.

7. Once the project has been created, open the ***xml***file and add the following configuration just below the ***properties*** element in the XML structure available.

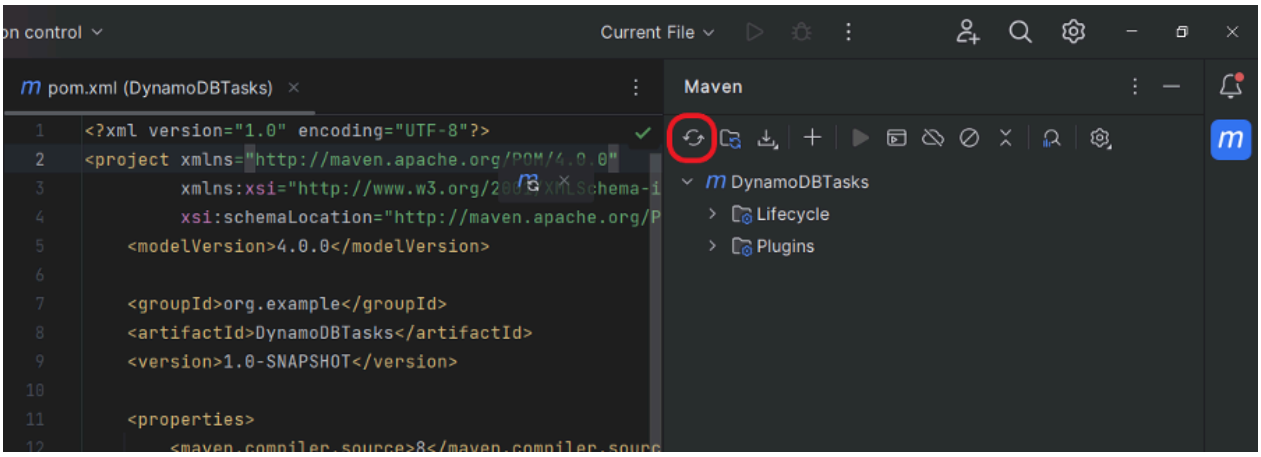
```
<dependencies>
    <dependency>
        <groupId>com.amazonaws</groupId>
        <artifactId>aws-java-sdk</artifactId>
        <version>1.12.546</version>
        <scope>compile</scope>
```

```
        </dependency>
    </dependencies>
```

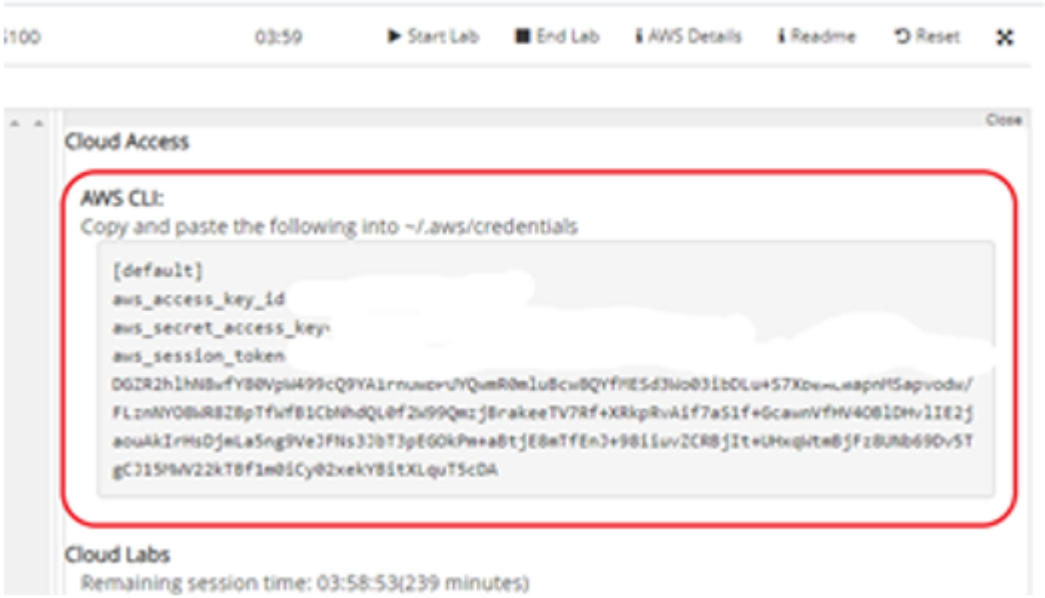
Your ***pom.xml*** should now look like what is shown in the image below:



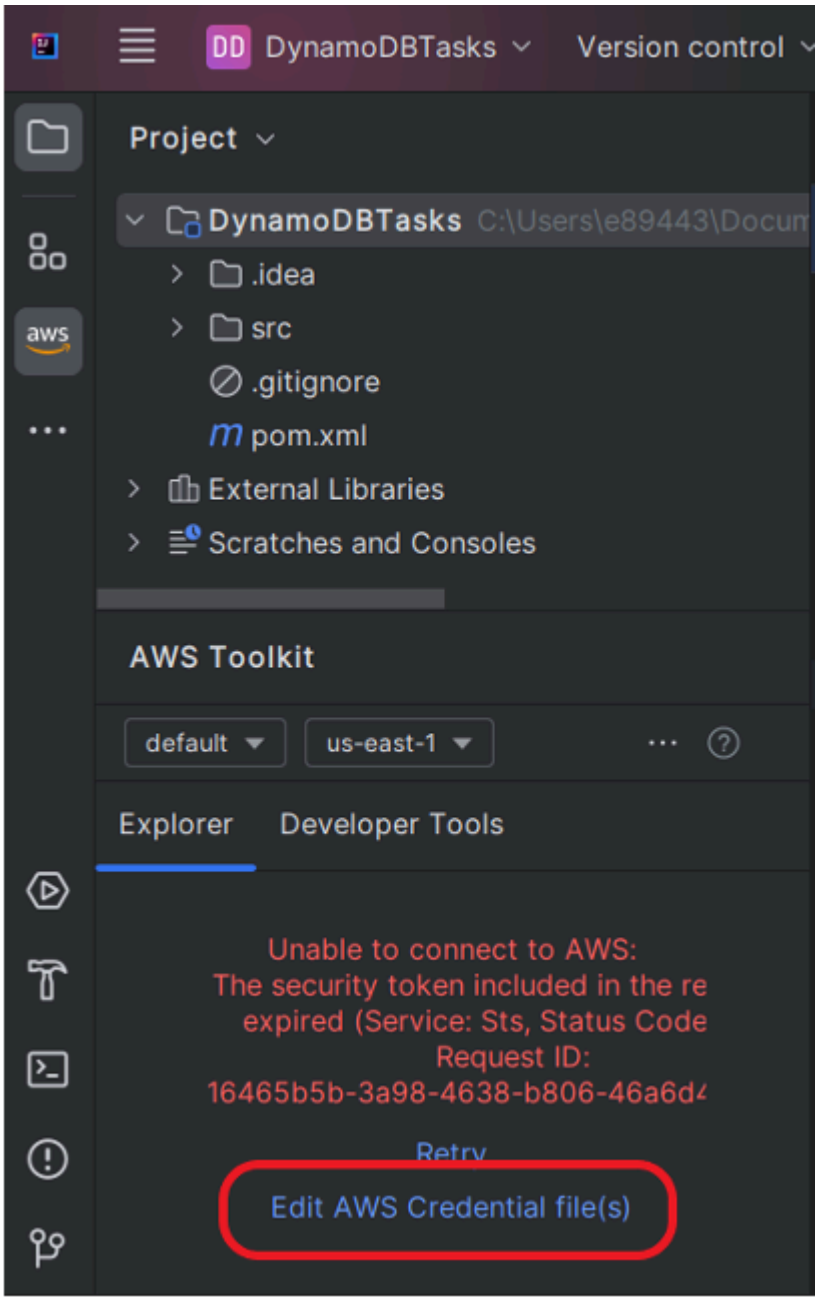
8. Now, resolve the maven configuration by tapping the **reload icon** shown below. This downloads the AWS SDK related artifacts from a [maven repository](#)(i.e. a place where the AWS SDK related library is kept as a maven artifact), and configures it in the Java project (i.e. **DynamoDBTasks**) you just created.



9. Grab your **Access Key Id**, **Secret Key**, as well as**Session Access Key** associated with your current AWS Academy portal, as below.



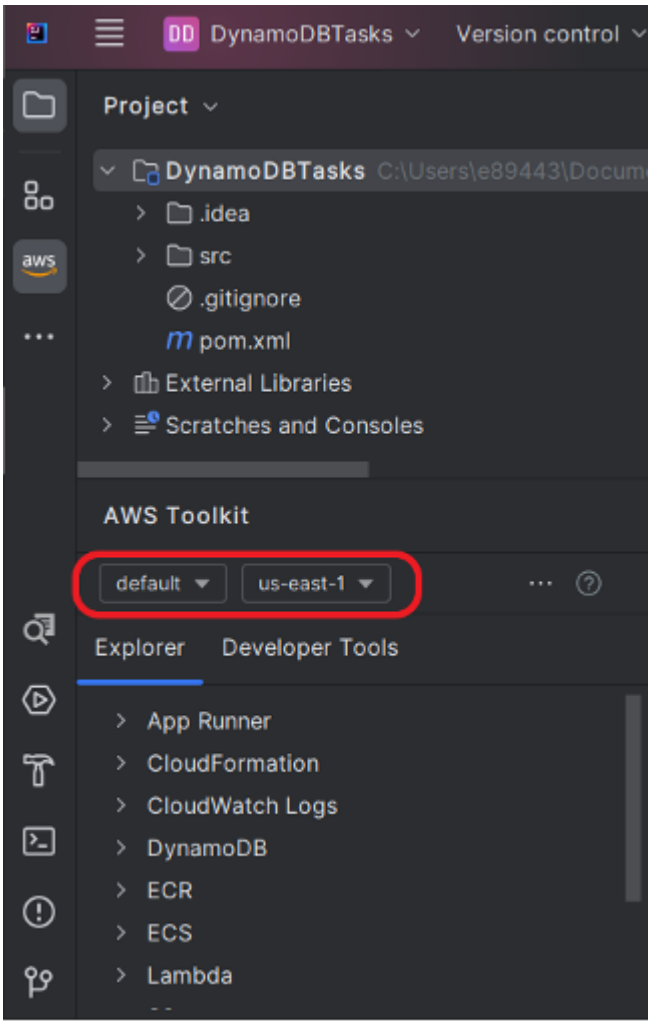
10. If you already had AWS credentials configured in IntelliJ IDEA before, you will now see the following error in your AWS Toolkit view.



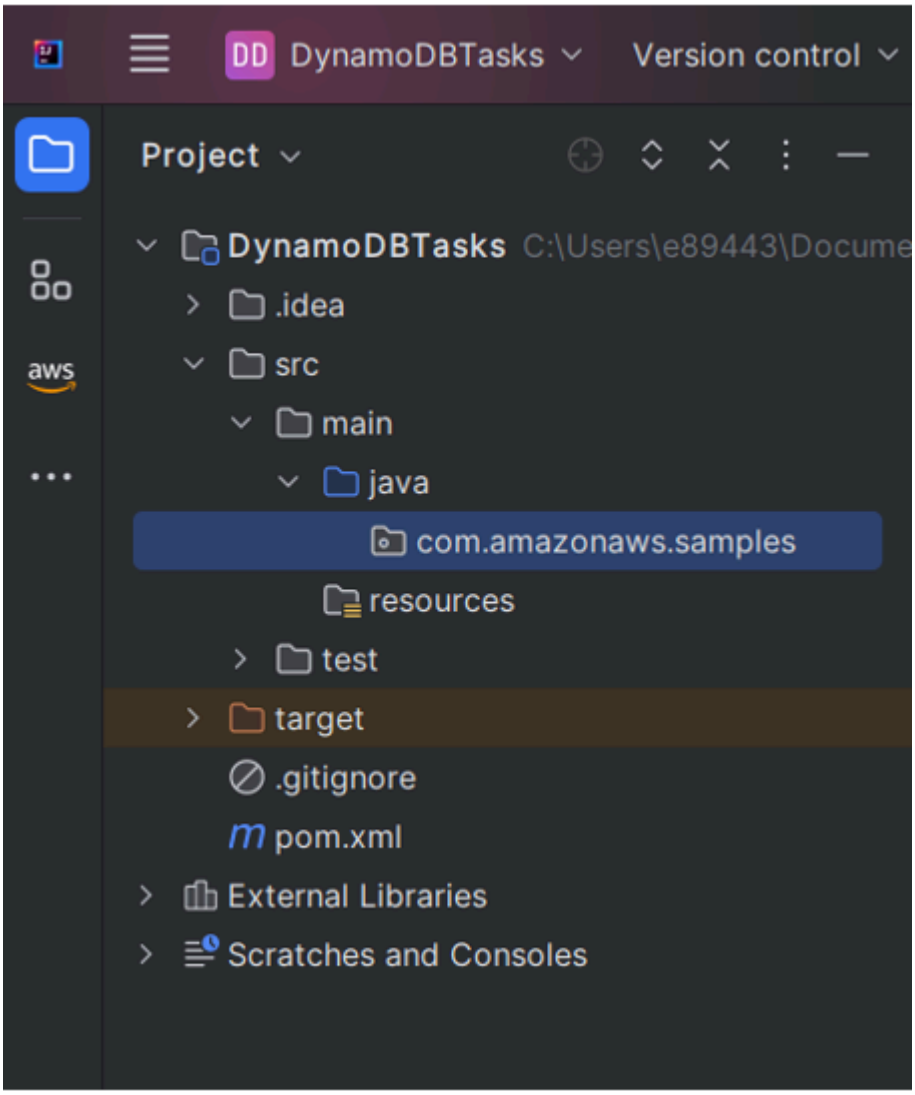
Tap **Edit AWS Credential file(s)** to update the AWS credentials with what you copied in the previous step.

Remember that the AWS Academy provisions short-lived credentials for us to work with, which expire when your lab session stops.

11. Once the AWS credentials have been updated, check whether the **default** credentials profile is selected. If not, use the relevant drop-down box in the AWS Toolkit view to select the **default** AWS credentials profile, as below.



12. Then, create a Java package named **com.amazonaws.samples** within the project you've created (i.e. **DynamoDBTasks**). Your project structure will now look like what is shown below.



You are now ready to use the project created (i.e. **DynamoDBTasks**) to add a DynamoDB table via AWS DynamoDB client APIs available in the AWS SDK for Java.

5.2 Create a DynamoDB Table

In this step, you will create a table named “Movies”. The primary key for the table is composed of the following attributes:

- year – The partition key. The `ScalarAttributeType` is N for number.
- title – The sort key. The `ScalarAttributeType` is S for string.

Step 1: Download **MoviesCreateTable.java** from the **DynamoDB-java.zip** source code archive, and paste it into your AWS Java project created previously under the package **com.amazonaws.samples**.

```
AmazonDynamoDB client = AmazonDynamoDBClientBuilder.stand
ard().withEndpointConfiguration(new AwsClientBuilder.Endp
```

```
ointConfiguration("http://localhost:8000", "us-east-1")).
build();
```

You set the endpoint to indicate that you are creating the table in DynamoDB on your computer.

```
Table table = dynamoDB.createTable(
    tableName,
    Arrays.asList(
        new KeySchemaElement("year", KeyType.HASH),
        new KeySchemaElement("title", KeyType.RANGE)),

    Arrays.asList(
        new AttributeDefinition("year", ScalarAttributeTy
pe.N),
        new AttributeDefinition("title", ScalarAttributeT
ype.S)),
    new ProvisionedThroughput(10L, 10L));
```

In the `createTable` call, you specify the desired table name, primary key attributes, and their corresponding data types.

The `ProvisionedThroughput` parameter is required, but the downloadable version of DynamoDB ignores it. (Provisioned throughput is beyond the scope of this exercise.)

Step 2: Compile and run the program.

5.3 Loading Data into DynamoDB

This task uses a sample data file that contains information about a few thousand movies from the Internet Movie Database (IMDb). The movie data is in **JSON** format, for each movie, there is a *year*, a *title*, and a JSON object named *info*.

```
{
    "year": 2013,
    "title": "Rush",
    "info": {
        "directors": ["Ron Howard"],
        "release_date": "2013-09-02T00:00:00Z",
        "rating": 8.3,
        "genres": [
            "Action",
            "Biography",
            "Drama",
            "Sport"
        ],
        "image_url": "http://ia.media-imdb.com/images/M/MV5BMTQyMDE0MTY0OV5BMl5BanBnXkFtZTcwMjI2OTI0OQ@@._V1_SX400_.jpg",
        "plot": "A re-creation of the merciless 1970s rivalry between Formula One rivals James Hunt and Niki Lauda.",
        "rank": 2,
        "running_time_secs": 7380,
        "actors": [
            "Daniel Bruhl",
            "Chris Hemsworth",
            "Olivia Wilde"
        ]
    }
},
```


In the JSON data, note the following:

- You use the **year and title** as the primary key attribute values for the **Movies**
- You store the **rest of the info values** in a single attribute called info.

Follow the instructions below to complete task.

Step 1: Download the sample data archive **moviedata.zip**, and then extract the datafile **moviedata.json** from the archive.

Step 2: Copy and paste **moviedata.json** into the home directory of the Java project created previously in IntelliJ IDEA.

Step 3: Download **MoviesLoadData.java** from the source code archive **DynamoDB-java.zip** and paste it into your AWS Java project created previously under the package **com.amazonaws.samples**.

Step 4: Compile and run the program.

Note: This program uses the open-source library **Jackson** to process JSON. **Jackson** is included in the AWS SDK for Java. You don't have to install it separately.

5.4 Create, Read, Update and Delete an Item

This task takes you through how to create, read, update, and delete an item in a DynamoDB table. Follow the instructions below.

Create a New Item – MoviesItemOps01.java

To add a new item to the Movies table. Download **MoviesItemOps01.java** from the source code archive **DynamoDB-java.zip** and paste it into your AWS Java project. Compile and run the program.

Note: The primary key is required. The sample code adds an item that has the primary key (year, title) and info attributes. The info attribute stores a JSON object that provides more information about the underlying movie.

Read an Item – MoviesItemOps02.java

In the previous program, you added the following item to the table:

```
{
    year: 2015,
    title: "The Big New Movie",
    info: {
        plot: "Nothing happens at all.",
        rating: 0
    }
}
```

You can use the `getItem` method to read the item from the **Movies** table. You must specify the primary key values, so you can read any item from **Movies** if you know its year and title.

Download **MoviesItemOps02.java** from the source code archive **DynamoDB-java.zip** and paste it into your AWS Java project. Compile and run the program.

Update an Item – MoviesItemOps03.java

In this task, you will perform the following updates:

- Change the value of the existing attributes (rating, plot).
- Add a new list attribute (actors) to the existing info.

You can use the `updateItem` method to modify an existing item. You can update values of existing attributes, add new attributes, or remove attributes.

The item changes from this:

```
{
    year: 2015,
    title: "The Big New Movie",
    info: {
        plot: "Nothing happens at all.",
```

```
        rating: 0
    }
}
```

To the following:

```
{
    year: 2015,
    title: "The Big New Movie",
    info: {
        plot: "Nothing happens all at once.",
        rating: 5.5,
        actors: ["Larry", "Moe", "Curly"]
    }
}
```

Download **MoviesItemOps03.java** from the source code archive **DynamoDB-java.zip** and paste it into your AWS Java project. Compile and run the program.

Please Note: This program uses an `UpdateExpression` to describe all updates you want to perform on the specified item.
The `ReturnValues` parameter instructs DynamoDB to return only the updated attributes (`UPDATED_NEW`).

Increment on Atomic Counter – MoviesItemOps04.java

DynamoDB supports atomic counters, where you use the `updateItem` method to increment or decrement the value of an existing attribute without interfering with other write requests. (All write requests are applied in the order in which they are received.)

The sample code shows how to increment the rating for a movie. Each time you run it, the program increments this attribute by one.

Download **MoviesItemOps04.java** from the source code archive **DynamoDB-java.zip** and paste it into your AWS Java project. Compile and run the program.

Update an Item Conditionally – MoviesItemOps05.java

The sample code shows how to use `UpdateItem` with a condition. If the condition evaluates to true, the update succeeds; otherwise, the update is not performed.

In this case, the movie item is updated only if there are more than three actors.

Download **MoviesItemOps05.java** from the source code archive **DynamoDB-java.zip** and paste it into your AWS Java project. Compile and run the program.

The program should fail with the following message:

```
The conditional request failed
```

This is because the movie has three actors in it, but the condition is checking for **greater than** three actors.

```
.withConditionExpression("size(info.actors) > :num")
```

Modify the program so that the `ConditionExpression` looks like this:

```
.withConditionExpression("size(info.actors) >= :num")
```

The condition is now **greater than or equal** *to* 3 instead of **greater than** 3.

Compile and re-run the program. The `UpdateItem` operation should now succeed.

Delete an Item – MoviesItemOps06.java

You can use the `deleteItem` method to delete one item by specifying its primary key. You can optionally provide a `ConditionExpression` to prevent the item from being deleted if the condition is not met.

Follow the instructions below to try to delete a specific movie item if its rating is 5 or less.

Download **MoviesItemOps06.java** from the source code archive **DynamoDB-java.zip** and paste it into your AWS Java project. Compile and run the program.

If you see the message `The conditional request failed`, please look at the conditions applied in the code and figure out why.

5.5 Query and Scan the Data

You can use the query method to retrieve data from a DynamoDB table.

Note: you must specify a partition key value. The sort key is optional.

The primary key for the **Movies** table is composed of the following:

- year – The partition key. The attribute type is number.
- title – The sort key. The attribute type is string.

To find all movies released during a year, you need to specify only the **year**. You can also provide the **title** to retrieve a subset of movies based on some condition (on the sort key); for example, to find movies released in 2014 that have a **title** starting with the letter "A".

In addition to **query**, there is also a **scan** method that can retrieve all the table's data.

Query Data – MoviesQuery.java

The code included in this step performs the following queries:

- Retrieve all movies released in the year.
- Retrieve all movies released in the *year*1992, with a *title* beginning with the letter "A" through the letter "L".

Step 1: Download **MoviesQuery.java** from the source code archive **DynamoDB-java.zip** and paste it into your AWS Java project.

In the sample code, note the following:

```
HashMap<String, String> nameMap = new HashMap<String, Str
ing>();nameMap.put("#yr", "year");
HashMap<String, Object> valueMap = new HashMap<String, Ob
ject>();
valueMap.put(":yyyy", 1985);
QuerySpec querySpec = new QuerySpec().withKeyConditionExp
ression("#yr = :yyyy").withNameMap(nameMap).withValueMap
(valueMap);
```

- `nameMap` provides name substitution. This is used because year is a reserved word in DynamoDB — you cannot use it directly in any expression, including `KeyConditionExpression`. You use the expression attribute name `#yr` to address this.
- `valueMap` provides value substitution. This is used because you can't use literals in any expression, including `KeyConditionExpression`.

You use the expression attribute `value:yyyy` to address this. First, you create the `querySpec` object, which describes the query parameters, and then you pass the object to the query method.

Step 2: Compile and run the program.

Please note: The preceding program shows how to query a table by its primary key attributes. In DynamoDB, you can optionally create one or more **secondary indexes** on a table and query those indexes in the same way that you query a table. Secondary indexes give your applications additional flexibility by allowing queries on non-key attributes.

Scan Data – MoviesScan.java

The scan method reads every item in the entire table and returns all the data in the table. You can provide an optional `filter_expression` so that only the items matching your criteria are returned. However, the filter is applied only after the entire table has been scanned.

The following program scans the entire **Movies** table, which contains approximately 5,000 items. The scan specifies the optional filter to retrieve only the movies **from the 1950s** (approximately 100 items) and discard all the others.

Step 1: Download **MoviesScan.java** from the source code archive **DynamoDB-java.zip** and paste it into your AWS Java project.

In the sample code, note the following:

```
ScanSpec scanSpec = new ScanSpec().withProjectionExpressi
on("#yr, title, info.rating").withFilterExpression("#yr b
etween :start_yr and :end_yr and title between :letter1 a
nd :letter2").withNameMap(new NameMap().with("#yr", "yea
r")).withValueMap(new ValueMap().withNumber(":start_yr",
1950).withNumber(":end_yr", 1959).withString(":letter
1","A").withString(":letter2","C"));
```

- `ProjectionExpression` specifies the attributes you want in the scan result.
- `FilterExpression` specifies a condition that returns only items that satisfy the condition. All other items are discarded.

Step 2: Compile and run the program.

Note: You can also use the *Scan* operation with any secondary indexes that you created on the table.

Activity 6: Working with AWS DynamoDB Remote APIs

In this task, you will revisit the same DynamoDB operations that you performed thus far. The only difference is that these operations are now performed on the DynamoDB tables provisioned in the **DynamoDB web service** rather than in **DynamoDB Local**. The code is generally same as what we looked at previously except for the settings of AWS region and credentials. You may directly replace code snippet that initialises an AWSDynamoDB client in all classes with the following.

```
AmazonDynamoDB client = AmazonDynamoDBClientBuilder.stand
ard().withRegion(Regions.US_EAST_1).withCredentials(new P
rofileCredentialsProvider("default"))  .build();
```

Note: AWS Academy ONLY allows you to perform these operations in **us-east-1** and **us-west-2** regions. Therefore, make sure you configure your `AWSDynamoDB client` to use one of the two regions. You will see AccessDenied exceptions being thrown around when you perform DynamoDB operations from your applications otherwise.